

JT_DEV_B73_1.0

Laravel Scrum Team Project

1 Initial situation

- 1.1 In your previous projects, you built applications with JavaScript, TypeScript, React, PHP, databases, and frameworks. You also already experienced teamwork and Scrum in a first simulation. This project is the end-year milestone, the technical challenge is real, but the main objective is to deliver software as a team.
- 1.2 You will work in a team of **4 to 6 apprentices**. Your team will probably include people with different levels of experience: not everyone will have completed exactly the same previous projects. This is intentional. In professional teams, you rarely work only with people who know exactly the same things as you. You will need to share knowledge, distribute work fairly, review each other's code, and make sure the whole team can understand the product.
- 1.3 The technology stack is fixed: you will build a web application with **Laravel**. This constraint is here to reduce time lost on technology choices and to let you focus on collaboration, delivery, product quality, and Scrum practice. You are expected to use Laravel conventions, Blade, Eloquent, migrations, validation, authorization.
- 1.4 The concrete product idea is not described in this brief. Your trainer will play the stakeholder/client role and will reveal the business need during the first stakeholder meeting. Your team **must contact the stakeholder at the start of the project** and schedule this first meeting, as in a real professional mandate.
- 1.5 Your team will use Scrum with **three-day** sprints. The team chooses who starts as Product Owner, Scrum Master, and Developers. Roles rotate every two sprints so that several apprentices practise coordination, backlog ownership, facilitation, and implementation responsibilities.
- 1.6 **Each sprint** must produce a **demonstrable increment**. At the end of every sprint, you present what works to the stakeholder and collect feedback. Every two sprints, when roles change, the team runs a retrospective and chooses concrete improvement actions for the next cycle.

2 Skills from the ICT training plan worked on

Skills ranked by importance of implementation in the project: [consult the training plan](#).

- 2.1 a1 : Clarifier et documenter les besoins des parties prenantes dans le cadre d'un projet ICT.
- 2.2 a4 : Planifier les projets ICT et les tâches selon un modèle de procédure.
- 2.3 a6 : Vérifier l'avancement des projets ICT et des tâches et en faire état selon le modèle de procédure.
- 2.4 a7 : Remettre la solution ICT au client et clôturer le projet.
- 2.5 g1 : Analyser et documenter les besoins en matière d'applications et d'interfaces.
- 2.6 g5 : Implémenter les applications et les interfaces selon le concept en respectant les exigences de sécurité.
- 2.7 g6 : Vérifier la qualité et la sécurité des applications et des interfaces.
- 2.8 c2 : Mettre en œuvre des modèles de données dans un dispositif de stockage de données numériques.
- 2.9 g2 : Vérifier la faisabilité technique des prototypes d'interfaces utilisateur et les développer.
- 2.10 g3 : Evaluer et documenter la sécurité des applications et des interfaces.

3 Deliverables

- 3.1 A single GitHub repository for the team, containing the full Laravel application source code.



- 3.2 A GitHub Project board containing the initial process tasks, the product backlog, sprint backlog items, and all team-created issues.
- 3.3 A working Laravel application that can be run locally from a clean clone by following the README instructions.
- 3.4 A README containing project description, setup instructions, seed data instructions, main features, and known limitations.
- 3.5 A `/docs` folder containing stakeholder meeting notes, role rotation history, sprint goals, sprint review summaries, retrospective summaries, architecture decisions.
- 3.6 A database schema implemented through Laravel migrations and documented with a simple diagram.
- 3.7 A final stakeholder demonstration.

4 Andragogic conditions

- 4.1 Work is carried out in teams of **4 to 6 apprentices**. The team owns delivery collectively, but each task must have a clear responsible person.
- 4.2 The team must organize knowledge sharing. Apprentices with stronger Laravel experience are expected to mentor others, not to implement everything alone.
- 4.3 You are encouraged to pair program on complex parts, especially architecture, authentication, authorization, data modelling, and debugging.
- 4.4 You have access to all necessary resources, especially official documentation, internal course material, and web resources.
- 4.5 Generative AI may be used to understand concepts, debug errors, compare implementation options, or improve explanations. It must not replace analysis, backlog decisions, team communication, code review, or your own understanding. Significant AI-generated code must be explained in the pull request.
- 4.6 The concrete stakeholder need must not be requested before the first stakeholder meeting. Your first responsibility is to contact the stakeholder and schedule this meeting.
- 4.7 The team uses Scrum with three-day sprints. Sprint Planning happens at the start of each sprint. Daily Scrum happens every work day. Sprint Review happens at the end of each sprint. Sprint Retrospective happens every two sprints, when roles change.
- 4.8 Product Owner and Scrum Master roles rotate every two sprints. The rotation must be documented in `/docs`.
- 4.9 **Maximum allowed time for the project: 18 full days (7.5 hours per day)**. The team must manage time, workload, stakeholder meetings, sprint ceremonies, and delivery autonomously. At the beginning of the project, inform your trainer of the team composition and planned start date.

5 Steps

- 5.1 Check you have access to the shared GitHub repo and project (The trainer will transmit them).
- 5.2 Contact the stakeholder and schedule the first meeting. Prepare questions before the meeting: goals, users, constraints, priorities, acceptance conditions, and out-of-scope elements.
- 5.3 Run the stakeholder kickoff meeting. Document the product vision, initial requirements, open questions, risks, and first acceptance criteria in `/docs`.
- 5.4 Decide the initial Scrum roles and document the role rotation plan for the whole project.
- 5.5 Build the first product backlog from stakeholder input. Write **user stories with acceptance criteria**, estimate them as a team, and prioritize them with the Product Owner.
- 5.6 Initialize the Laravel project, development environment, database, formatting tools, test setup, and README. Make sure every team member can run the app locally.



- 5.7 Model the first version of the database and application structure. Validate it with the stakeholder or trainer before implementing large parts of the product.
- 5.8 Run three-day sprint cycles. At the start of each sprint, define a sprint goal and select a realistic sprint backlog. During the sprint, work through branches, pull requests, reviews, and regular integration.
- 5.9 Demonstrate a working increment to the stakeholder at the end of every sprint. Collect feedback and update the product backlog.
- 5.10 Before the final demo, stabilize the product: fix critical bugs, complete documentation, verify setup instructions, run tests, clean the backlog, and prepare handover.
- 5.11 Run the final stakeholder demonstration and project closure. Present what was delivered, what remains open, known limitations, and what the team learned.

6 Resources

- 6.1 Scrum Guide: <https://scrumguides.org/scrum-guide.html>
- 6.2 Agile Manifesto: <https://agilemanifesto.org>
- 6.3 User stories and acceptance criteria: <https://www.atlassian.com/agile/project-management/user-stories>
- 6.4 Linking pull requests to issues: <https://docs.github.com/en/issues/tracking-your-work-with-issues/linking-a-pull-request-to-an-issue>
- 6.5 Laravel: <https://laravel.com/docs/13.x/installation>
- 6.6 Team work: <https://docs.in.jt-lab.ch/apprentis/cours/projets/team-work>
- 6.7 Project management: <https://docs.jt-lab.ch/apprentis/cours/projets/project-management>
- 6.8 Git repository configuration: <https://docs.in.jt-lab.ch/dev/git-repo-configuration>
- 6.9 Working with Git: <https://docs.in.jt-lab.ch/dev/working-with-git>
- 6.10 Conventional commits: <https://docs.in.jt-lab.ch/dev/commits-naming-convention>

7 Evaluation criteria

- 7.1 Stakeholder collaboration:
 - The team contacted the stakeholder autonomously.
 - Requirements were clarified, documented, and kept up to date.
 - Sprint reviews included working demonstrations and stakeholder feedback.
- 7.2 Scrum process:
 - Each sprint had a clear sprint goal and realistic sprint backlog.
 - Daily Scrums happened every work day and blockers were visible.
 - Retrospectives happened every two sprints and led to concrete improvement actions.
 - Scrum roles rotated every two sprints and responsibilities were actually practised.
- 7.3 Backlog and planning quality:
 - User stories are understandable, prioritized, and have acceptance criteria.
 - Work is sliced into small deliverable increments.
 - GitHub Project reflects real progress and remains readable throughout the project.
- 7.4 Collaboration quality:
 - Work distribution is fair and transparent despite different experience levels.
 - Pull requests show constructive reviews, knowledge sharing and real analysis.
 - The repository history shows regular collaboration from several team members.
- 7.5 Laravel product quality:



- The application runs locally from a clean clone following the README.
- Laravel conventions are respected for routing, controllers, models, views, form validation, authorization, migrations...
- Database relations are coherent and implemented through migrations.
- Forms validate input server-side and show clear errors.
- Authentication and authorization are secure and appropriate for the product.

7.6 Code and delivery quality:

- Formatting and linting are applied consistently.
- Code structure respects Laravel MVC model and good programming patterns (Kiss, DRY, single responsibility).
- Commits follow conventional commits.
- Each significant task is implemented through a pull request linked to the issue.
- No secrets are committed to the repository.
- There is no dead code.

7.7 Final handover:

- Final demo shows a coherent product increment.
- README and `/docs` provide enough information for the stakeholder/trainer to understand, run, and continue the project.
- Known limitations and unfinished work are documented honestly.

